

THINSTATION-NG

Compilar desde Ubuntu o Linux Mint Sustitución de la Opción A (Fedora nativo) usando Docker

■ **Por qué funciona con Docker:** el script **setup-chroot** de ThinStation-NG crea internamente un entorno chroot basado en Fedora. Si ejecutamos ese script *dentro de un contenedor Fedora*, obtenemos exactamente el entorno requerido, independientemente del sistema operativo del host (Ubuntu, Mint, Debian...).

Complemento del tutorial:	ThinStation-NG Tutorial RDP Selectivo
Versión ThinStation-NG:	7.2-Stable
Sistema host válido:	Ubuntu 22.04/24.04, Linux Mint 21/22
Herramienta:	Docker Engine (no Docker Desktop)
Fecha:	22 de May de 2026

1. Cómo funciona setup-chroot con Docker

El script **setup-chroot** (280 líneas de bash, léible en el repositorio oficial) hace exactamente esto:

state_checks()

Comprueba si `var/cache/dnf` existe. Si no, llama a `ts/bin/install_chroot` que descarga y despliega el entorno Fedora base usando DNF.

do_mounts()

Monta `/dev`, `/proc`, `/sys`, `/tmp` y `/run` dentro del directorio del proyecto usando `mount --bind`. Esto requiere privilegios de root y capacidades Linux equivalentes a `--privileged` en Docker.

chroot "\$PWD" /ts/TS_ENV

Entra en el chroot Fedora y lanza el entorno de compilación de ThinStation. Todo el proceso DNF, compilación y generación de imágenes ocurre dentro.

Conclusión: el host solo necesita poder ejecutar un contenedor Fedora con acceso completo a las syscalls de montaje. Docker con **--privileged** proporciona exactamente eso desde cualquier distribución Linux.

Comparativa: host Fedora vs Docker en Ubuntu

	Host Fedora nativo	Docker en Ubuntu/Mint
SO del host	Fedora obligatorio	Ubuntu, Mint, Debian, cualquiera
DNF disponible	Sí (nativo)	Sí (dentro del contenedor Fedora)
mount --bind	Sí (root nativo)	Sí (Docker --privileged)
chroot	Sí (root nativo)	Sí (dentro del contenedor)
ISO resultante	Idéntica	Idéntica
Aislamiento	Bajo (modifica el host)	Alto (todo en el contenedor)

3. Workflow completo: clonar → configurar → compilar

El proceso completo desde Ubuntu/Mint se divide en tres fases:

FASE 1	Preparar el directorio en el host Ubuntu/Mint
FASE 2	Ejecutar el contenedor Fedora con el repo montado como volumen
FASE 3	Dentro del contenedor, configurar y compilar

FASE 1 — Preparar el directorio en el host (Ubuntu/Mint)

```
# En una terminal de Ubuntu/Mint (como usuario normal, NO root)

# 1. Clonar el repositorio ThinStation-NG
git clone --depth 1 https://github.com/thinstation/thinstation-ng.git
cd thinstation-ng

# 2. Crear los archivos de configuración personalizados
#   (antes de lanzar el contenedor, para que Docker los vea)

# build.conf – copiar el ejemplo base y editar
#   (el archivo build.conf.example estará disponible DENTRO del contenedor
#   en /build/ tras ejecutar setup-chroot por primera vez)
# Por ahora lo dejamos vacío; lo editaremos dentro del contenedor.

# thinstation.conf.buildtime – configuración de sesión
cat > thinstation.conf.buildtime << 'EOF'
NET_USE_DHCP=On
NET_USE=BOTH
TIME_ZONE=Europe/Madrid
KEYMAP=es
SESSION_0_TYPE=rdp-selector
SESSION_0_AUTOSTART=On
RECONNECT_PROMPT=On
EOF

# Verificar que estamos en el directorio correcto
pwd
ls -la setup-chroot
```

FASE 2 — Lanzar el contenedor Fedora

Ejecutamos un contenedor Fedora con el repositorio montado como volumen. El flag **--privileged** es imprescindible porque setup-chroot usa **mount --bind** para /dev, /proc, /sys, etc.

```
# Aún en el host Ubuntu/Mint, dentro del directorio thinstation-ng/

# Lanzar contenedor Fedora interactivo:
# --privileged → necesario para mount --bind (setup-chroot lo requiere)
# -v "$PWD":/ts-ng → monta el repo en /ts-ng dentro del contenedor
# --name tsng-build → nombre del contenedor para referencia futura
# -it fedora:latest → imagen Fedora oficial, modo interactivo

docker run --privileged \
    -v "$PWD":/ts-ng \
    --name tsng-build \
    -it fedora:latest \
    /bin/bash

# El prompt cambiará a algo como: [root@<hash> /]#
# Ya estás DENTRO del contenedor Fedora.
```

■ Si el contenedor ya existe de una sesión anterior, reconéctate con:

```
docker start -ai tsng-build
```

FASE 3 — Dentro del contenedor Fedora

```
# ■■■ Dentro del contenedor Fedora ■■■■
```

```
# 1. Instalar git en el contenedor
dnf install -y git

# 2. Ir al directorio del repositorio montado
cd /ts-ng

# 3. Inicializar el entorno chroot de ThinStation
# PRIMERA VEZ: descarga los paquetes Fedora necesarios (~10-15 min)
./setup-chroot
# Cuando termine, el prompt será del entorno chroot de ThinStation.
# Pulsa 'q' para salir del README si aparece.

# 4. Ir al directorio de compilación (dentro del chroot de TS)
cd /build

# 5. Preparar build.conf
rm -f build.conf
cp build.conf.example build.conf
# Editar build.conf con vi o nano:
vi build.conf
# Asegúrate de descomentar/añadir:
# package freerdp
# package dialog
# package xterm
# package rdp-selector

# 6. Copiar thinstation.conf.buildtime (ya lo pusiste en /ts-ng/)
cp /ts-ng/thinstation.conf.buildtime /build/thinstation.conf.buildtime

# 7. Crear el paquete personalizado rdp-selector
mkdir -p /build/packages/rdp-selector/bin
mkdir -p /build/packages/rdp-selector/etc/cmd

# (Copia el contenido del script de la Sección 7 del tutorial principal)
# El archivo fuente está disponible desde el host en:
# thinstation-ng/rdp-selector.sh (si lo creaste antes)
# O créalo directamente:
vi /build/packages/rdp-selector/bin/rdp-selector.sh
vi /build/packages/rdp-selector/etc/cmd/rdp-selector

# Dar permisos de ejecución
chmod +x /build/packages/rdp-selector/bin/rdp-selector.sh
chmod +x /build/packages/rdp-selector/etc/cmd/rdp-selector

# 8. Compilar la imagen
./build
# Primera compilación: 15-30 minutos según velocidad de CPU/red.
```

4. Extraer la ISO al host Ubuntu/Mint

Al usar `-v "$PWD":/ts-ng`, el volumen Docker está montado bidireccionalmente. Pero la compilación ocurre en `/build` (dentro del chroot de ThinStation, que está dentro del contenedor). Necesitamos copiar la ISO al volumen compartido:

```
# ■■ Aún dentro del contenedor, en el chroot de ThinStation ■■■■■■■■■■

# Verificar que la ISO se generó correctamente
ls -lh /build/boot-images/grub/
# Deberías ver: thinstation.iso y el directorio boot/

# Copiar la ISO y los archivos PXE al volumen compartido con el host
cp /build/boot-images/grub/thinstation.iso /ts-ng/
cp -r /build/boot-images/grub/boot/ /ts-ng/boot-pxe/

# Salir del chroot de ThinStation
exit

# Salir del contenedor Fedora
exit
```

De vuelta en el host Ubuntu/Mint, la ISO ya estará en el directorio **thinstation-ng/**:

```
# En el host Ubuntu/Mint, en el directorio thinstation-ng/
ls -lh thinstation.iso
ls -lh boot-pxe/

# La ISO está lista para:
# 1. Copiarse a un USB con Ventoy (ver Sección 10 del tutorial principal)
# 2. Usarse para PXE (ver Sección 11 del tutorial principal)
```

✓ La ISO generada en el contenedor es idéntica a la que se obtendría en un host Fedora nativo. El contenedor Docker solo actúa como capa de compatibilidad para ejecutar el entorno Fedora requerido.

6. Resolución de problemas frecuentes

■ Error: 'permission denied' al ejecutar setup-chroot dentro del contenedor

Asegúrate de que el contenedor se lanzó con `--privileged`. Sin este flag, `mount --bind` falla con error de permisos. Elimina el contenedor y vuelve a crearlo con el flag correcto: `docker rm tsng-build` `docker run --privileged -v "$PWD":/ts-ng --name tsng-build -it fedora:latest /bin/bash`

■ Error: 'You have already started the session' al ejecutar setup-chroot

El script detecta que ya hay un chroot activo. Sal completamente del chroot anterior con `'exit'` y vuelve a ejecutar `setup-chroot`.

■ El contenedor no arranca porque ya existe

Si ves `'Error: Conflict. The container name tsng-build is already in use'`, usa: `docker start -ai tsng-build` para reconectar al existente, o `docker rm tsng-build` para eliminarlo y crear uno nuevo.

■ La ISO no aparece en el host tras la compilación

Recuerda copiarla al volumen compartido antes de salir del chroot: `cp /build/boot-images/grub/thinstation.iso /ts-ng/` Los archivos en `/build` solo existen dentro del chroot del contenedor.

■ Error de red durante dnf dentro del contenedor

Verifica que Docker tiene acceso a internet: `docker run --rm fedora:latest dnf install -y git` Si falla, revisa las reglas de firewall del host o el DNS de Docker (archivo `/etc/docker/daemon.json` con `dns: ["8.8.8.8"]`).

■ Linux Mint: el codename del repositorio Docker no se encuentra

Linux Mint usa `UBUNTU_CODENAME` en `/etc/os-release` en lugar de `VERSION_CODENAME`. El comando de instalación de la Sección 2 ya gestiona esto con: `$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")`

